

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 12
“Searching”



DISUSUN OLEH:
Muhammad Nabil Raissa P
108092500127
S1IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

Berikut ini merupakan materi lanjutan untuk kasus pencarian data. Selain pencarian nilai ekstrim, kita terkadang juga mencari suatu data yang lebih spesifik. Misalnya mencari mahasiswa dengan nama atau nim tertentu, mencari rumah dengan alamat tertentu, dan lainnya. Berbeda dengan pencarian nilai ekstrim, yang mana nilai yang dicari selalu ditemukan, maka pada kasus pencarian ini terdapat kemungkinan bahwa data yang dicari tidak ditemukan. Selain itu pada kasus pencarian ini akan diperkenalkan algoritma pencarian yang memanfaatkan keterurutan data.

12.1 Sequential Search

Pencarian secara sekuensial ini adalah pencarian yang dilakukan dari data pertama, kedua hingga terakhir secara satu persatu dan berurutan. Ciri khas dari pencarian ini adalah proses pencarian akan berhenti ketika data yang dicari ditemukan, walaupun masih terdapat data yang belum dicek nilainya. Algoritma ini dikenal dengan nama ***Sequential Search***, karena prosesnya melakukan pengecekan setiap elemen array secara satu persatu dan sekuensial dari data pertama hingga ditemukan atau data terakhir.

- 1) Asumsi terdapat suatu array of integer T dengan indeks dari 0 hingga N-1, dan suatu nilai yang dicari pada array T, yaitu X.
- 2) Status pencarian digunakan untuk menandakan data yang dicari ditemukan atau tidak, misalnya found dengan tipe boolean.
- 3) Pencarian dilakukan dari T[0] sampai ke T[N-1], setiap kali perbandingan dengan X, update nilai found.
- 4) Perulangan harus dihentikan apabila status pencarian found bernilai true (data ditemukan) atau T[N-1] telah dicek.

SOAL LATIHAN

Statement perulangan

1.

Source Code:

```
package main

import (
    "fmt"
)

func main() {
    var vote int
    var totalVotes int
```

```

var validVotes int
var counts [21]int

for {
    fmt.Scan(&vote)
    if vote == 0 {
        break
    }
    totalVotes++
    if vote >= 1 && vote <= 20 {
        validVotes++
        counts[vote]++
    }
}

fmt.Printf("Suara masuk: %d\n", totalVotes)
fmt.Printf("Suara sah: %d\n", validVotes)
for i := 1; i <= 20; i++ {
    if counts[i] > 0 {
        fmt.Printf("%d : %d\n", i, counts[i])
    }
}
}

```

Output:

The screenshot shows the Visual Studio Code interface with a Go file named `1.go` open. The code is a voting system. The output window shows the results of the program execution:

```

Suara sah: 8
1 : 1
2 : 1
3 : 2
7 : 1
18 : 1
19 : 2

```

Deskripsi Program:

Kode program Go tersebut berfungsi untuk membaca data nomor urut pilihan warga secara terus-menerus dalam sebuah perulangan hingga ditemukan angka 0 sebagai penanda akhir data. Di dalam perulangan, program otomatis menghitung total seluruh suara yang masuk dan melakukan validasi untuk memastikan nomor pilihan berada di rentang 1 hingga 20. Jika suara dinyatakan valid, program akan meningkatkan jumlah suara sah dan menjumlahkannya ke dalam array penghitung berdasarkan nomor calon ketua RT masing-masing. Setelah proses input selesai, program akan langsung menampilkan informasi total suara masuk, total suara sah, serta rincian perolehan suara untuk setiap calon ketua RT yang mendapatkan minimal satu suara secara berurutan.

2.

Source Code:

```
package main

import (
    "fmt"
)

func main() {
    var vote int
    var totalVotes int
    var validVotes int
    var counts [21]int

    for {
        fmt.Scan(&vote)
        if vote == 0 {
            break
        }
        totalVotes++
        if vote >= 1 && vote <= 20 {
            validVotes++
            counts[vote]++
        }
    }

    fmt.Printf("Suara masuk: %d\n", totalVotes)
    fmt.Printf("Suara sah: %d\n", validVotes)

    max1 := 0
    max2 := 0
    ketua := 0
    wakil := 0

    for i := 1; i <= 20; i++ {
        if counts[i] > max1 {
            max2 = max1
```

```

        wakil = ketua
        max1 = counts[i]
        ketua = i
    } else if counts[i] > max2 {
        max2 = counts[i]
        wakil = i
    }
}

fmt.Printf("Ketua RT: %d\n", ketua)
fmt.Printf("Wakil ketua: %d\n", wakil)
}

```

Output:

```

func main() {
    // main redeclared in this block
    validVotes++
    counts[vote]++
}

fmt.Printf("Suara masuk: %d\n", totalVote)
fmt.Printf("Suara sah: %d\n", validVotes)

max1 := 0
max2 := 0
ketua := 0
wakil := 0

for i := 1; i <= 20; i++ {
    if counts[i] > max1 {
        max2 = max1
        wakil = ketua
        max1 = counts[i]
        ketua = i
    }
}

fmt.Printf("Ketua RT: %d\n", ketua)
fmt.Printf("Wakil ketua: %d\n", wakil)
}

```

```

18
19
0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19
PS C:\Users\nabil\OneDrive\Dokumen\semester2alpro>

```

Deskripsi Program:

Jadi program ini bakal *looping* input buat ngitung suara RT sampe nemu angka 0.

Suara yang sah (1–20) disimpan di array, terus di-scan buat nyari juara 1 dan 2. Nah, karena nyari juaranya urut dari nomor kecil ke gede dan pake perbandingan >, pas ada suara seri otomatis nomor urut terkecil yang menang jadi Ketua atau Wakil. Terakhir, tinggal print total suara sama pemenangnya.

3.

Source Code:

```

package main

import "fmt"

```

```

const NMAX = 1000000
var data [NMAX]int

func main() {
    var n, k int
    fmt.Scan(&n, &k)

    isiArray(n)

    hasil := posisi(n, k)

    if hasil == -1 {
        fmt.Println("TIDAK ADA")
    } else {
        fmt.Println(hasil)
    }
}

func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {
    left := 0
    right := n - 1

    for left <= right {
        mid := left + (right-left)/2
        if data[mid] == k {
            return mid
        } else if data[mid] < k {
            left = mid + 1
        } else {
            right = mid - 1
        }
    }

    return -1
}

```

Output:

